SmartWaste Pro: Code Guide (file by file, function by function)

මේ ගයිඩ් එකෙන් project එකේ code එක **මුල ඉඳන් අග දක්වා** තේරුම් ගන්න පුළුවන්. හැම ෆයිල් එකකම: **ඒක මොකටද**, ඒකේ **ප්‍රධාන functions මොනවද**, ඒවා **මොනවද කරන්නේ**.

කියවන අනුපිළිවෙල: 1 (මුලින් app එක පටන් ගන්න විදිහ) → 2 (Firebase සම්බන්ධය) → 3 (screens, user journey එක අනුව) → 4 (shared කොටස්) → 5 (rules) → 6 (admin site) → 7 (hardware data).

---

0. Big picture (මුලින්ම)

```
ESP32 bin ──Wi-Fi──► Realtime Database (IoT Firebase project)
                              │ (live read)
                ┌─────────────┴─────────────┐
     Mobile app (Expo)                Admin site (React)
                └─────────────┬─────────────┘
                              ▼
            Main Firebase project: Auth, Firestore, Storage, Functions
```

- **Mobile app** = `mobile/` (React Native + TypeScript). Seller සහ Buyer දෙන්නාම මේ app එකම, role එක අනුව වෙනස් screens.
- **Admin site** = `admin/` (React + Vite, JavaScript).
- **දත්ත** Firestore collections 5ක: `users`, `marketplace`, `orders`, `offers`, `notifications`.
- **ආරක්ෂාව** = `mobile/firestore.rules` (database එකේම නීති).
- **Server එකක් නෑ.** App එක කෙළින්ම Firebase එකට කතා කරනවා.

---

1. App එක පටන් ගන්න විදිහ (Navigation)

Expo Router **ෆයිල් නම්වලින්ම screens හදනවා**: `app/` ඇතුළේ ෆයිල් එකක් = screen එකක්.

`mobile/app/_layout.tsx`: root layout

- **`RootLayout()`**: මුළු app එකම `SafeAreaProvider` (notch/පහළ පට්ටියට ඉඩ) සහ navigation theme එකක් ඇතුළේ තියනවා. Theme එකේ background එක අපේ `Palette.background` (ලා කොළ), ඒ නිසා screens මාරු වෙද්දී කළු flash එකක් එන්නේ නෑ. Stack navigator එකෙන් screens වලට "slide from right" animation එකක් දෙනවා.

`mobile/app/(tabs)/_layout.tsx`

- **`TabLayout()`**: `(tabs)` folder එකේ screens සඳහා Stack එක (swipe back ඉඩ දෙන්න).

`mobile/app/(tabs)/index.tsx`: Splash (පළමු screen එක)

- **`Splash()`**: logo එක fade + rise animation එකකින් පෙන්නලා, ටිකක් ඉන්නකොට `router.replace('/(tabs)/Welcome')` කරනවා. `useEffect` එක animation එක ආරම්භ කරලා timer එකක් දානවා.

`mobile/app/(tabs)/Welcome.tsx`

- **`Welcome()`**: "Get Started" බොත්තම එබුවාම `router.push('/(tabs)/Login')`.

`mobile/app/(tabs)/Login.tsx`: Login

- **`handleLogin()`**:
  1. email/password හිස්ද කියලා බලනවා.
  2. `signInWithEmailAndPassword(auth, email, password)`: Firebase Auth එකෙන් login.
  3. `getDoc(doc(db, "users", user.uid))`: ඒ user ගේ profile එක Firestore එකෙන් ගන්නවා (**role** එක ඒකේ).
  4. `userData.disabled` නම් signOut කරලා "Account disabled" කියනවා.
  5. `role === 'seller'` නම් `SellerDashboard` එකට, නැත්නම් `BuyerDashboard` එකට `router.replace`.
  6. වැරදි නම් "Invalid email or password" Alert එකක්.
- **`handleForgotPassword()`**: email එක ගහලා "Forgot password?" එබුවාම `sendPasswordResetEmail`. කිසිම email එකකටත් එකම පණිවිඩය පෙන්නනවා (`passwordResetMessage`), ඒ නිසා කාගේ ගිණුම් තියෙනවද කියලා හොයන්න බෑ.

`mobile/app/(tabs)/CreateAccount.tsx`: Sign up

- **`handleContinue()`**: නම, email, password, confirm password හිස්ද / ගැලපෙනවද බලනවා → `createUserWithEmailAndPassword` (Auth එකේ ගිණුම) → `setDoc(doc(db,"users",user.uid), {fullName, email, role, createdAt})` (Firestore profile) → role අනුව dashboard එකට.
- **role තේරීම:** `ROLES` array එක (Seller / Buyer) card දෙකකින් පෙන්නනවා. Admin කෙනෙක් මෙතනින් හදන්න බෑ (rules වලින් නවත්තලා).
- `mismatch` = password දෙක නොගැලපෙනවා නම් රතු පණිවිඩයක්.

---

2. Firebase සම්බන්ධය සහ shared data types

`mobile/firebaseConfig.js`

- Firebase **main project** එකට සම්බන්ධ වෙනවා (`initializeApp`).
- `auth = initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage) })`: **login එක phone එකේ මතක තියාගන්නවා** (app එක ආයේ ඇරියාම login නැතුව ඇතුළු වෙන්න).
- Export කරන්නේ: `auth`, `db` (Firestore), `storage` (profile photos), `functions` (Stripe cloud functions).
- Keys `.env` වලින් (`process.env.EXPO_PUBLIC_FIREBASE_*`).

`mobile/firebaseConfig.web.js`

- ඒ වගේම, web build එකට (persistence වෙනස්).

`mobile/iotConfig.js`: smart bin එකේ Firebase project එක

- වෙනම **IoT Firebase project** එකකට සම්බන්ධ වෙනවා (`getDatabase` = Realtime Database). Export: `iotDb`.
- `signInAnonymously`: නම නැතුව (anonymous) sign in වෙලා bin data read කරන්න.
- `DEFAULT_BIN_ID = 'bin001'`.
- `USE_MOCK_IOT`: dev mode එකේ විතරක් (`__DEV__`) bin නැති වෙලාවට **බොරු data** (`MOCK_BIN_DATA_NORMAL`, `MOCK_BIN_DATA_ALERT`) පෙන්වන්න. Release build එකේ කවදාවත් නෑ.

`mobile/types.ts`: TypeScript types

`UserProfile`, `MarketplaceItem` (listing), `Order`, `Offer`, `AppNotification`, `BinData`/`BinCompartment`, `UserLocation`, `WasteType`. මේවා නිසා කෝඩ් එකේ field නම් වැරදුණොත් editor එකම කියනවා.

`mobile/hooks/useAuthUser.ts`: වැදගත්ම hook එකක්

- `useAuthUser()`: login වුණු user ව දෙනවා, **ඒත් ආරක්ෂිතව**:
  - Firebase login එක phone එකේ මතකයෙන් ආපහු ගන්නකල් (`onAuthStateChanged`) බලාගෙන ඉන්නවා (නැත්නම් "Missing or insufficient permissions" error එකක් එනවා).
  - `firebaseUser.reload()` කරලා ඒ ගිණුම තාමත් තියෙනවද බලනවා. මකලා නම් / disable කරලා නම් `signOut` කරලා Login එකට යවනවා.
  - Login වෙලා නැත්නම් `router.replace('/(tabs)/Login')`.
- සියලුම screens මේක පාවිච්චි කරන්නේ Firestore listeners පටන් ගන්න කලින් user ඉන්නවද බලන්න.

---

3. Screens (user journey එක අනුව)

3A. Seller

`seller/SellerDashboard.tsx`: Seller ගේ Home (ලොකුම screen එකක්)

මොකද්ද පෙන්නන්නේ: Bin එකේ compartments තුන (live), Bin ON/OFF, donut chart එක, bin location map, notifications, issue report.

State: `binData` (bin එකේ දත්ත), `freshness` (ON/OFF), `notifications`, `sellerData` (profile), `placeQuery/placeState` (bin location search).

`useEffect`s: 1. **Bin data listener:** `onValue(dbRef(iotDb, "bins/<binId>"), ...)`: Realtime Database එකෙන් bin එකේ දත්ත ආපු ගමන් `normalizeBin(...)` (utils) කරලා `setBinData`. `binId` = `sellerData.binId || 'bin001'`. 2. **Freshness tick:** තත්පර 15කට සැරයක් "දැන් ON ද OFF ද" ආයේ ගණනය (`describeFreshness`). 3. **User profile + notifications listener:** `onSnapshot` users doc එකටයි notifications (`toUid == uid`) වලටයි. Bin alert notifications (`bin_full`) පෙරලා ඉවත් කරනවා. 4. **Place search (type-ahead):** `placeQuery` වෙනස් වුණාම මිලි 450කින් පස්සේ `searchPlaces(text)` (utils/placeSearch). පරණ ප්‍රතිඵල අලුත් අකුරුවලට පෙන්වන්නේ නෑ (`placeState.text` ගැලපෙනවද

බලනවා). 5. **Listings sync:** bin location එක ඇති නම් Seller ගේ **available listings** වල `location` එක bin location එකට මාරු කරනවා (`getDocs` + `writeBatch`), එවිට Buyer ගේ route එක නිවැරදි.

Functions: - **`handleSelectPlace(place)`**: Seller suggestion එකක් තෝරාගත්තාම `updateDoc(users/uid, { binLocation, binPlaceName })`. - **`handleReportSubmit()`**: "Report an issue" sheet එකෙන් කෙටි පණිවිඩයක් (දැනට Alert). - **`markAllAsRead()`**: notifications bell එක එබුවාම unread ඔක්කොම `read: true`. - **`BinCard`** (component): compartment එකකට card එක (%, progress bar, status pill, kg, overweight ලේබලය, moisture). - **Overweight banner:** `overweightLabels` (compartments වලින් `overweight` true ඒවා) තියෙනවා නම් රතු Notice එකක්. - Notification එකක් tap කළාම `notificationRoute(type,'seller')` අනුව අදාළ screen එකට `router.push`.

`seller/AddWaste.tsx`: waste listing හදන screen

- `useEffect`s: profile (name, binId, binLocation) ගන්නවා, bin data subscribe කරනවා, phone GPS (fallback) ගන්නවා.
- **`getCompartment/availableKg/fillLevel(type)`**: bin එකේ ඒ compartment එකේ දැන් තියෙන බර/ මට්ටම.
- **`handleWeightChange(type, value)`**: අකුරු පෙරලා (අංක විතරයි), `total = weight × pricePerKg` ගණනය කරලා `roundMoney` කරනවා.
- **`handleSubmit()`**: තෝරපු compartments සඳහා:
  1. බර bin එකේ තියෙන බරට වැඩිද (`maxWeight`) බලනවා ("Capacity exceeded").
  2. Bin location එක තියෙනවද බලනවා (profile `binLocation` මුලින්, නැත්නම් GPS).
  3. හැම compartment එකකටම `addDoc(collection(db,'marketplace'), {sellerUid, sellerName, wasteType, weightKg, pricePerKg, totalPrice, status:'available', location, createdAt})`.
- `priceConfig`: වර්ගයකට kg එකක මිල (plastic, food, metal).

`seller/SellerOrders.tsx`: Seller ගේ orders

- `useEffect`: `orders` where `sellerUid == uid`, `createdAt desc` (`onSnapshot`).
- **Tabs:** `OrderTabs` + `tabForOrder/countByTab` (utils/orderTabs): Ongoing / Completed / Cancelled.
- **`run(order, work, failMessage)`**: ඕනම action එකක් `busyId` සමඟ ආරක්ෂිතව ධාවනය කරනවා (loading + error Alert).
- **`notifyBuyer(order, event)`**: Buyer ට `order_confirmed / order_completed / order_cancelled` notification එකක් (`buyerNotificationFor`).
- **`handleConfirm`**: status → `confirmed` + buyer ට notification.
- **`handleComplete`**: cash order නම් `paymentStatus: 'paid'` මුලින් (cash ගත්තා), ඊට පස්සේ status → `completed` + notification.
- **`handleDecline`**: `writeBatch`: order → `cancelled` **සහ** listing එක ආපහු `available` (`soldOrderId` මකලා), එකම පියවරකින්. Buyer ට notification.
- **`OrderCard`**: order විස්තර, Buyer ගේ rating තරු, `sellerActionsFor(order)` අනුව බොත්තම් (Confirm/Decline/Complete).

`seller/SellerOffers.tsx`: offers

- `useEffect`: `offers` where `sellerUid == uid` (withdrawn පෙරලා), pending ඉස්සරහට, ඉහළම මිල උඩට.
- **`handleAccept(offer)`**: `runTransaction` එකකින් (එකම පියවරක්):

1. listing තවම available ද, offer තවම pending ද බලනවා.
2. offer → `accepted`, listing → `sold` (`soldOrderId`), අලුත් **order** එක (`status:'pending'`, `awaitingBuyer:true`, Buyer දැම්ම මිලට).
3. ඊට පස්සේ අනිත් pending offers `rejected` + notifications. Buyer ට "accepted" notification.

- **`handleReject(offer)`**: offer → `rejected` + notification.

`seller/SellerProfile.tsx`

- `useEffect`: profile + Seller ගේ completed orders වලින් **earned** සහ **weight** එකතු කරනවා (`formatKgTotal`, `formatRs`).
- **`handleLogout()`**: `signOut(auth)` → Login.

`seller/EditProfile.tsx`

- පේළි 5ක wrapper: `<ProfileEditor variant="seller" />`.

3B. Buyer

`buyer/BuyerDashboard.tsx`: Marketplace (Buyer ගේ Home)

- `useEffect`: GPS ගන්නවා (දුර ගණනය කරන්න), `marketplace` where `status == 'available'` (`onSnapshot`).
- **`filteredItems`**: search text + type filter (All/Plastic/Food/Metal).
- **`getDistance(location)`**: `calculateDistance` (Haversine).
- **`ListingCard`**: listing card (මිල, බර, kg මිල, දුර, Route / Buy Now / Make an Offer).
- **`handleMakeOffer` / `handleSubmitOffer`**: මිල validate කරලා (`offeredPrice` අංකයක්, 0ට වැඩි) `addDoc(offers)` + Seller ට `offer_received` notification.
- **`handleBuyNow`** → payment sheet (cash / card) → **`handleConfirmOrder`**:
  - **`confirmPurchase(method, cardLast4)`**: **`writeBatch`**: order හදනවා + listing → `sold` (`soldOrderId`) එකම පියවරේ. Seller ට `order_placed` notification. අසාර්ථක නම් (වෙන කෙනෙක් කලින් ගත්තා නම්) පැහැදිලි පණිවිඩයක්.
  - **`handleCardPayment`**: `createCheckoutSession` (Cloud Function) → Stripe page එක `expo-web-browser` එකෙන් → `verifyCheckoutSession` (server එකෙන් ගෙවුවද තහවුරු කරලා card අංක 4 ලබාගන්නවා).
- **`openRouteMap(item)`**: GPS අවසරය → දුර → map modal එක → `fetchRoadRoute` (utils/route) background එකේ. පාර ලැබුණාම Polyline එක පාරවල් අනුව, නැත්නම් කෙළින් ඉර.

`buyer/BuyerOrders.tsx`

- `useEffect`: `orders` where `buyerUid == uid`; Tabs (Ongoing / Completed / Cancelled).
- **`handleCancelOrder`**: order → cancelled + listing ආපහු available (batch) + Seller ට notification.
- **`handlePayOrder`**: accepted-offer order එකක් card එකෙන් ගෙවනවා (Stripe) → `paymentMethod:'card', paymentStatus:'paid', paymentLast4`.
- **`handleOrderMap`**: "View location" map + route.
- **`handleRate(order, n)`**: completed order එකකට `rating` (1-5) එක පාරක්.
- `OrderCard`: විස්තර, `orderCode(order.id)` (ORD-...), Pay by card, Cancel, rating.

`buyer/BuyerOffers.tsx`

- Listener දෙකක්: Buyer ගේ **offers**, සහ Buyer ගේ **orders** (offer එකක් පිළිගත්නද, order එක මොකක්ද කියලා `offerOrders[offerId]`).
- Pending / Offer history කොටස් දෙකක්. Pending එකට "Waiting for the seller..." (Withdraw නෑ).
- **`handleBuy(offer, order)`**: Seller පිළිගත්නාට පස්සේ order → `confirmed`, `awaitingBuyer:false` + Seller ට notification.
- **`handleDecline(offer, order)`**: order cancel + listing relist (batch) + notification.

`buyer/BuyerProfile.tsx`, `buyer/EditProfile.tsx`

- Profile stats (orders ගණන, බර), `IdBadge` (USR-...), logout. `EditProfile` = `ProfileEditor variant="buyer"`.

3C. `components/profile-editor.tsx`: profile edit (දෙපැත්තටම)

- **`handlePhoneChange/handleLocationChange`**: input පෙරලීම.
- **`handlePickImage`**: `expo-image-picker` එකෙන් photo එකක්.
- **`uploadImage(uri)`**: Firebase **Storage** එකට upload → URL එක `photoURL`.
- **`handleSave`**: `updateDoc(users/uid, {fullName, phone, location, photoURL, updatedAt})`.

---

4. Shared කොටස්

4A. `mobile/utils/`: pure functions (tests ඇත)

ෆයිල්	Functions	මොකටද
`binTelemetry.ts`	`parseLevel`, `parseWeightKg`, `parseOverweight`, `normalizeBin`, `toEpochMs`, `describeFreshness`, `formatAge`	Firmware එකේ අමු data ("50%", grams, true/false) ඇප් එකට හරියන අංක බවට (level %, status, kg). Bin ON/OFF (2 min)
`orderActions.ts`	`sellerActionsFor`, `completingCollectsCash`	order status අනුව Seller කරන්න පුළුවන් බොත්තම්
`orderTabs.ts`	`tabForOrder`, `countByTab`	order එක Ongoing/Completed/Cancelled කොයි tab එකද
`orderNotifications.ts`	`buyerNotificationFor`	Seller ගේ ක්‍රියාවට Buyer ට යන notification (type + පණිවිඩය)
`notificationRoute.ts`	`notificationRoute`	notification එක tap කළාම යන screen එක
`donut.ts`	`donutSlices`, `formatKg`, `formatKgTotal`	donut chart කොටස් ජ්‍යාමිතිය, බර පෙන්වීම
`money.ts`	`roundMoney`, `formatRs`	මිල සත 2කට වට, "Rs 7.20"
`distance.ts`	`calculateDistance`	Haversine දුර (km)
`route.ts`	`fetchRoadRoute`, `parseRoute`	OSRM පාර route

ෆයිල්	Functions	මොකටද
`placeSearch.ts`	`searchPlaces`, `parsePlaces`	Photon type-ahead ස්ථාන search
`passwordReset.ts`	`passwordResetMessage`	reset email පණිවිඩ
`shortId.ts`	`userCode`, `orderCode`	USR-..., ORD-... කෙටි ID

4B. `mobile/components/ui/`: UI කොටස්

`screen.tsx` (`Screen`, `ScreenHeader`, `keyboardAware`), `card.tsx` (`Card`, `SectionTitle`, `Divider`, `DetailRow`), `button.tsx`, `text-field.tsx` (password show/hide), `badge.tsx` (`Badge`, `Notice`, status පාට/නම්), `chip.tsx`, `sheet.tsx` (bottom sheet, `avoidKeyboard`), `bottom-nav.tsx`, `empty-state.tsx`, `list-option.tsx`, `donut-chart.tsx`, `order-tabs.tsx`, `star-rating.tsx`, `notification-bell.tsx`, `id-badge.tsx`. `platform-map.tsx` (+ `.web.tsx`): map එක device එකේ `react-native-maps`, web එකේ පණිවිඩයක්.

4C. `mobile/constants/`

- `design.ts`: **Design tokens**: `Palette` (පාට), `Space`, `Radius`, `Shadow`, `Type` (අකුරු ප්‍රමාණ), `wasteAccent(type)` (plastic නිල්, food කොළ, metal තැඹිලි).
- `bin-status.ts`: firmware status (EMPTY/LOW/HALF/75%/FULL/ERROR) → පාට, නම, `isBinFull`.

---

5. Cloud Functions: `mobile/functions/index.js`

- **`createCheckoutSession`**: Buyer ගෙවන මුදලට Stripe Checkout session එකක් හදලා URL එක දෙනවා (server එකේ secret key එකෙන්).
- **`verifyCheckoutSession`**: session එක ඇත්තටම ගෙවලාද (server එකෙන්ම) සහ ඒ Buyer ගේමද බලලා card last4 දෙනවා.
- (deploy කරලා නෑ; Blaze plan + Stripe key ඕන.)

---

6. Security rules: `mobile/firestore.rules`

Database එකේම ආරක්ෂාව. වැදගත් functions: - `isSignedIn`, `isOwner`, `isAdmin` (users doc එකේ `role == 'admin'`). - `isOrderParty`, `isOfferParty`: notification එකක් යවන්නේ ඒ order/offer එකේ දෙපැත්තෙන් එකෙන් විතරයි. - `isBuyingListing`, `isRestoringListing`: listing එක sold කරන්නේ **order එක හදන එකම batch එකේ**; relist කරන්නේ order cancel කරන batch එකේ. - `isAcceptedOfferOrder`: Seller offer accept කරද්දී හදන order එක offer එකට ගැලපෙන්න ඕන (මිල, Buyer), `status:'pending'`, `awaitingBuyer:true`. - `isValidNewListing/isValidListingEdit`: waste type, බර/මිල හරි සීමා. - `isValidBinLocation`, `isValidAdminProfileEdit`: coordinates / admin edit වලංගුද. - Collection එකක නීති: `users` (පෞද්ගලික), `marketplace` (ඕනම login user බලන්න, Seller හදන්න), `orders` (පාර්ශව දෙක + admin), `offers`, `notifications`. - **Tests:** `mobile/tests/rules/*.test.mjs` (127). Emulator එකෙන් "මේක කරන්න පුළුවන්/බෑ" සත්‍යාපනය. - `firestore.indexes.json`: query වලට අවශ්‍ය composite indexes 3ක්.

---

7. Admin site: `admin/src/`

- `main.jsx` → `App.jsx`: `Gate()` = Login නැත්නම් `Login`, admin නොවේ නම් `NotAdmin`, නැත්නම් `Layout` + routes (`/`, `/bins`, `/users`, `/marketplace`, `/orders`).
- `AuthContext.jsx` + `useAuth.js`: login වුණු user සහ `isAdmin` (users doc එකේ role).
- `firebaseConfig.js`, `iotConfig.js`: main project සහ IoT project (mobile වගේම).
- **Pages:**
  - `Overview.jsx`: KPI cards (users, listings, orders, revenue).
  - `BinStatus.jsx`: `BinStatus` (සියලුම bins list, search, ON/OFF filter) + `BinDetail` (එක bin එකක live readings).
  - `Users.jsx`: users table (admin ලා උඩින්), `openEdit/saveEdit` (නම, phone, location, points), `assignBin` (seller ගේ `binId`), search by `USR-`.
  - `Marketplace.jsx`: `Marketplace` (sellers list) + `SellerListings` (ඒ seller ගේ listings, Remove).
  - `Orders.jsx`: `Orders` (users list) + `UserOrders` (ඒ user ගේ orders, status වෙනස).
  - `Layout.jsx`, `Login.jsx`, `NotAdmin.jsx`.
- **`lib/`:**
  - `binTelemetry.js`: mobile වගේම (normalize, `normalizeBins`, `summarizeBin`).
  - `filters.js`: `filterUsers`, `filterListings`, `filterOrders`, `filterBins`, `groupListingsBySeller`, `groupOrdersByUser`, `filterPeople`, `orderUsersAdminsFirst`.
  - `userEdit.js`: edit form validate. `shortId.js`: USR-/ORD- (mobile වගේම).
  - `usePaginatedCollection.js`: 25 බැගින් live listener.

---

8. එක flow එකක් මුල ඉඳන් අග දක්වා (උදාහරණය: Seller waste විකුණනවා, Buyer ගන්නවා)

1. **App ඇරෙනවා:** `index.tsx` (Splash) → `Welcome` → `Login`.
2. **Seller login:** `handleLogin` → role `seller` → `SellerDashboard`.
3. **Bin data:** ESP32 → Realtime DB → `SellerDashboard` ගේ `onValue` → `normalizeBin` → cards/donut.
4. **Listing:** `AddWaste.handleSubmit` → `marketplace` එකට `addDoc` (status `available`).
5. **Buyer:** `BuyerDashboard` ගේ `onSnapshot` listing එක පෙනෙනවා → Route (`openRouteMap`) → Buy Now → `confirmPurchase` (batch: order + listing sold).
6. **Seller:** `SellerOrders` ගේ listener order එක පෙන්නනවා → `handleConfirm` → Buyer ට notification → `handleComplete` (cash ගත්තා, completed).
7. **Buyer:** `NotificationBell` එකෙන් notification එක tap → `BuyerOrders` → `handleRate` (තරු).
8. **Admin:** admin site එකේ Orders → ඒ user ගේ order එක, Marketplace → seller ගේ listing, Users → `USR-…` ID එකෙන් හොයනවා.

අතරතුර හැම database ලිවීමක්ම `firestore.rules` වලින් පරීක්ෂා වෙනවා (වැරදි වැඩ ප්‍රතික්ෂේප).

ඉක්මන් මතක තියාගන්න

ප්‍රශ්නය	පිළිතුර
Login මතක තියාගන්නේ කොහොමද?	`firebaseConfig.js` හි AsyncStorage persistence
Listing එක දෙපාරක් විකුණන්න බැරි ඇයි?	Buy කරද්දී order + listing sold එකම `writeBatch` එකේ; rules වලින් එය නහවුරු
Bin ON/OFF කොහොමද?	`lastUpdated` තත්පර 120ට වඩා පරණ නම් OFF (`describeFreshness`)
Overweight කොහොමද?	Firmware එකේ `overweight: true` ; `normalizeBin` ඒක ගන්නවා; banner පෙන්නනවා
Admin කෙනෙක් හදන්නේ?	Firebase console එකෙන් `users/{uid}.role = 'admin'` අතින්
Short ID එක කොහෙන්ද?	Firestore document ID එකේ පළමු අකුරු 6 (`shortId.ts`)
Card ගෙවීම්?	Cloud Functions + Stripe (deploy කරලා නෑ; demo එකට cash)